

```
In [120... import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import KFold
```

Question 1: Training and test error under a fixed design

```
In [51]: def question1(repetitions,n,p):
        """
        nsim : number of simulation
        n: number of sample
        p : number pf covariate
        """
        rng = np.random.default_rng(43202)

        #generate X_all data drawn from N(0,1) and intercept
        X = rng.normal(0, 1, size=(n, p))
        X_all = np.column_stack((np.ones(n), X))

        sigma = 1.0
        covariate_count = np.arange(p + 1)

        #let  $\beta_j=0.4j$ 
        beta = np.zeros(p + 1)
        beta[1:] = 0.4 ** np.sqrt(np.arange(1, p + 1))

        mu = X_all @ beta

        train_mse_decay = np.empty((repetitions, p+1))
        test_mse_decay = np.empty((repetitions, p+1))
        for re in range(repetitions):
            ## generate training and test reponse
            y_train = mu + rng.normal(loc=0.0, scale=sigma, size=n)
            y_test = mu + rng.normal(loc=0.0, scale=sigma, size=n)
            ## fit for the intercep model
            y_hat = np.repeat(y_train.mean(), n)
            train_mse_decay[re, 0] = np.mean((y_train - y_hat) ** 2)
            test_mse_decay[re, 0] = np.mean((y_test - y_hat) ** 2)
            for m in range(1, p+1):
                X_m = X_all[:, :m+1]
                beta_hat = np.linalg.lstsq(X_m, y_train, rcond=None)[0]
                y_hat = X_m @ beta_hat
                train_mse_decay[re, m] = np.mean((y_train - y_hat) ** 2)
                test_mse_decay[re, m] = np.mean((y_test - y_hat) ** 2)

        bias_mse_decay = np.empty(p + 1)
        bias_mse_decay[0] = np.mean((mu - (mu).mean()) ** 2)

        for m in range(1, p + 1):
            X_m = X_all[:, :m + 1]
            beta_hat = np.linalg.lstsq(X_m, mu, rcond=None)[0]
            y_hat = X_m @ beta_hat
            bias_mse_decay[m] = np.mean((mu - y_hat) ** 2)

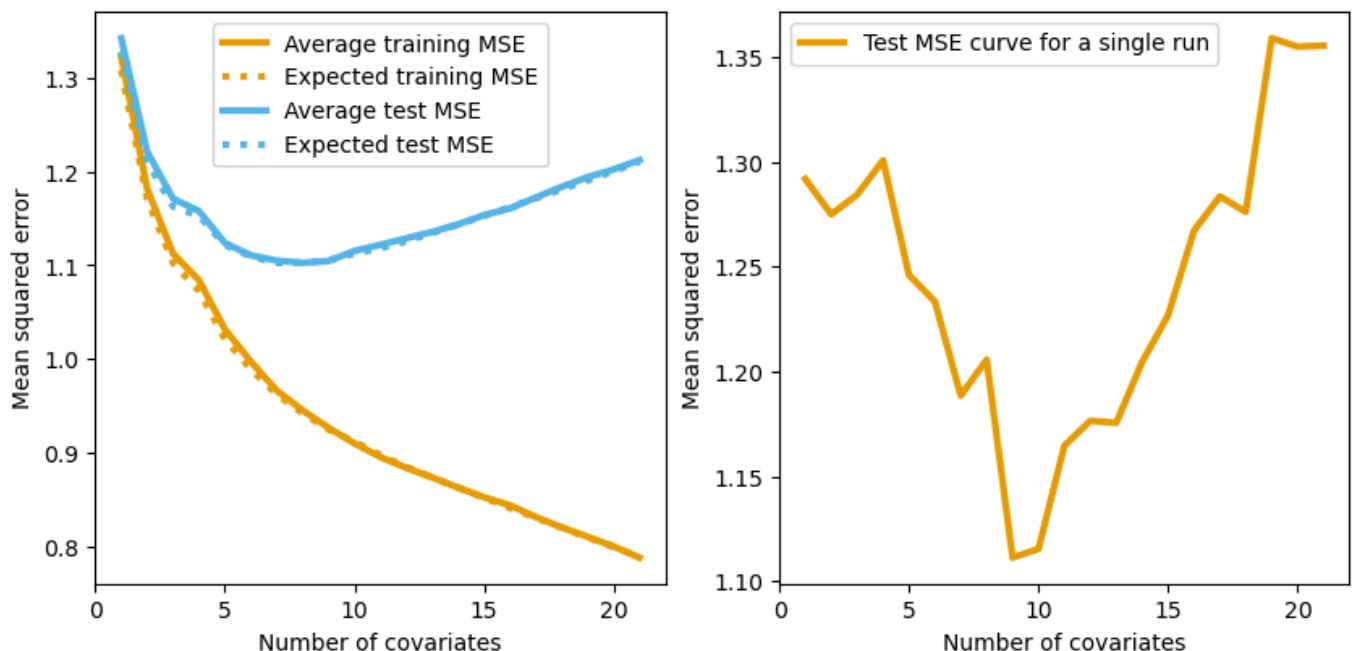
        theory_train_mse_decay = bias_mse_decay + (1 - (covariate_count + 1) / n) * sigma**2
        theory_test_mse_decay = bias_mse_decay + (1 + (covariate_count + 1) / n) * sigma**2
        return train_mse_decay,test_mse_decay,theory_train_mse_decay,theory_test_mse_decay
```

```
In [52]: train_mse,test_mse,expected_train_mse,expected_test_mse = question1(200,100,20)
```

```
In [56]: def plot_question_result(train_mse,test_mse,expected_train_mse,expected_test_mse):
fig,axs = plt.subplots(1,2,figsize=(10,5))
for ax_id,ax in enumerate(axs.flat):
    ax.set_box_aspect(1)
    ax.set_xlabel("Number of covariates")
    ax.set_ylabel("Mean squared error")

    if ax_id ==0:
        mean_train_mse_decay = train_mse.mean(axis=0)
        mean_test_mse_decay = test_mse.mean(axis=0)
        ax.plot(np.arange(1,train_mse.shape[1]+1),mean_train_mse_decay,color="#E69F00")
        ax.plot(np.arange(1,train_mse.shape[1]+1),expected_train_mse,color="#E69F00")
        ax.plot(np.arange(1,train_mse.shape[1]+1),mean_test_mse_decay,color="#56B4E9")
        ax.plot(np.arange(1,train_mse.shape[1]+1),expected_test_mse,color="#56B4E9")
        ax.legend(loc="best")
    else:
        ax.plot(np.arange(1,train_mse.shape[1]+1),test_mse[5],color="#E69F00",lw=2)
        ax.legend(loc="best")
```

```
In [57]: plot_question_result(train_mse,test_mse,expected_train_mse,expected_test_mse)
```



As the number of covariates increases, both the average training MSE and the expected training MSE show a continuous downward trend. However, the average test MSE and the expected test MSE exhibit a U-shaped trajectory, first decreasing and then increasing, with the test error reaching its minimum when the number of covariates is approximately 6; beyond that number, the test error gradually rises. In addition, the expected MSE curves closely match the corresponding average MSE curves on both the training and test sets.

c.1 Random noise fluctuations are averaged out in the average curve, but a single run preserves the random variation inherent in a single response vector.

c.2

$$E(\text{MSE}_{\text{train}} | X) = \frac{B^2}{n} + \left(1 - \frac{p}{n}\right) \sigma^2$$

where:

- B^2 is the sum of squared model biases (i.e., the sum of squared differences between the fitted function and the true function; if the model is correctly specified, then $B^2 = 0$);
- p the total number of model parameters;
- n is the sample size;
- σ^2 is the variance of the error term. As the number of predictors p increases:

(1) The variance term $(1 - \frac{p}{n}) \sigma^2$ monotonically decreases (because $1 - \frac{p}{n}$ becomes smaller).

(2) The bias term $\frac{B^2}{n}$ does not increase, since adding more variables may capture additional true signal, and even if they are not, the bias at least remains unchanged.

Question 2: Prediction error at one target point

In [132...

```
def question2(repetitions, n,p):
    sigma = 1.0
    rng = np.random.default_rng(43203)
    i = np.arange(1, n + 1)[: , None]
    j = np.arange(1, p + 1)[None, :]
    X_all = np.sqrt(2) * np.cos(np.pi * j * (i - 0.5) / n)
    beta = np.array([0.5,0.5,0.5,0.5,0.5,0.5],dtype=float)
    x0 = np.array([0,1,0,0,1,0])

    mu = X_all @ beta
    mu0 = x0 @ beta

    simulation_error = np.empty((repetitions, p + 1))

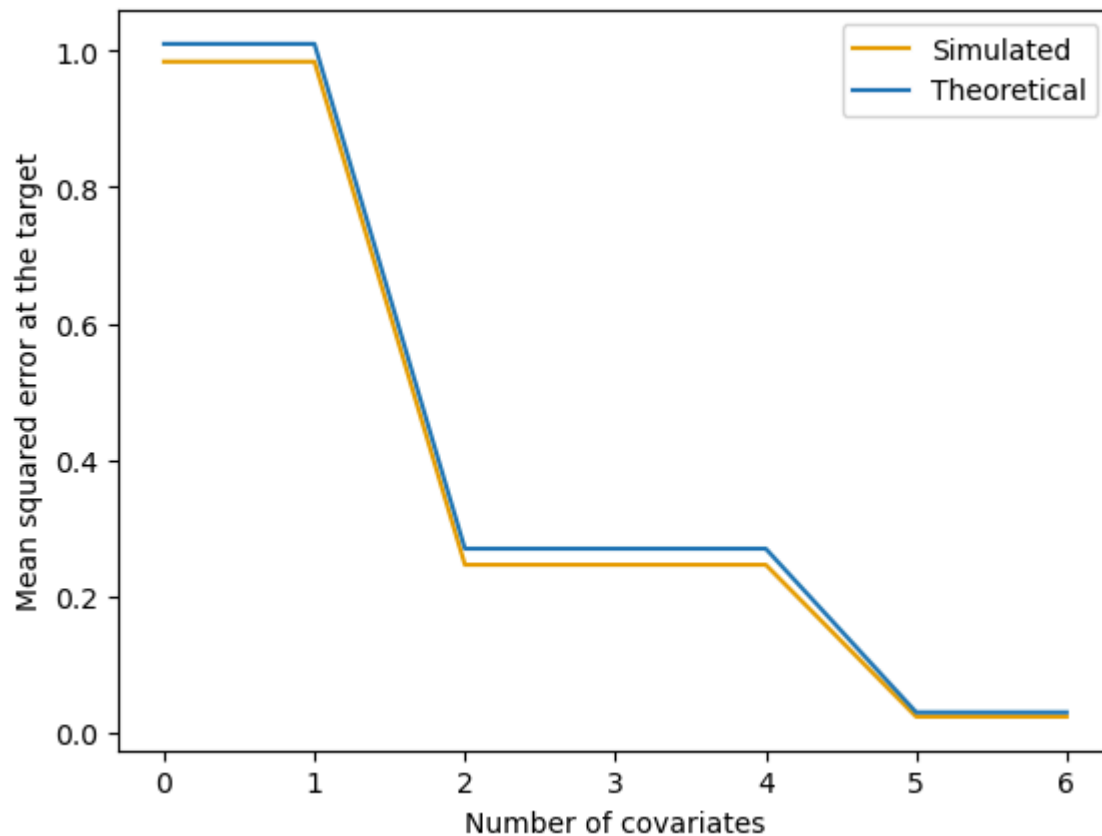
    for k in range(repetitions):
        y_train = mu + rng.normal(size=n)
        for m in range(p + 1):
            X_m = np.column_stack((np.ones(n), X_all[:, :m]))
            beta_hat = np.linalg.lstsq(X_m, y_train, rcond=None)[0]
            target_row = np.r_[1, x0[:m]]
            mu0_hat = target_row @ beta_hat
            simulation_error[k, m] = (mu0_hat - mu0) ** 2
    simulated = simulation_error.mean(axis=0)
    theoretical = np.empty(p + 1)

    for m in range(p + 1):
        omitted = x0[m:] @ beta[m:]
        est_var = sigma**2 * (1 + np.sum(x0[:m] ** 2)) / n
        theoretical[m] = omitted**2 + est_var
        print("m = %d simulated = %.4f theoretical = %.4f"
              % (m, simulated[m], theoretical[m]))
    fig,ax = plt.subplots(1,1)
    ax.plot(np.arange(p + 1),simulated,"#E69F00",label = "Simulated")
    ax.plot(np.arange(p + 1),theoretical,label = "Theoretical")
    ax.legend(loc="best")
    ax.set_xlabel("Number of covariates")
    ax.set_ylabel("Mean squared error at the target")
```

In [133...

```
question2(200,100,6)
```

m = 0	simulated = 0.9839	theoretical = 1.0100
m = 1	simulated = 0.9839	theoretical = 1.0100
m = 2	simulated = 0.2464	theoretical = 0.2700
m = 3	simulated = 0.2464	theoretical = 0.2700
m = 4	simulated = 0.2464	theoretical = 0.2700
m = 5	simulated = 0.0237	theoretical = 0.0300
m = 6	simulated = 0.0237	theoretical = 0.0300



a. The expected squared error equals the squared bias plus the variance. The bias arises because model m omits the last $p-m$ predictors and so fails to fit their contributions at the target point; the variance arises because the $m+1$ coefficients must be estimated from noisy data, and this estimation uncertainty is propagated to the prediction through x_0 .

b. Smallest m that minimizes the theoretical error is 5.

c. Only predictors 2 and 5 influence μ_0 because they are the only coordinates where x_0 is nonzero, whereas Question 1's test MSE averages over all rows so every predictor contributes.

Question 3: The optimism correction

1. Derive Expected Training MSE, Expected Test MSE, and the Expected Optimism

(1) Expected Training MSE

We have

$$\begin{aligned} \mathbf{y} - \mathbf{H}\mathbf{y} &= \boldsymbol{\mu} + \boldsymbol{\epsilon} - \mathbf{H}(\boldsymbol{\mu} + \boldsymbol{\epsilon}) \\ &= (\mathbf{I}_n - \mathbf{H})\boldsymbol{\mu} + (\mathbf{I}_n - \mathbf{H})\boldsymbol{\epsilon}. \end{aligned}$$

Taking the squared norm and expectation, the cross term

$$2(\mathbf{I}_n - \mathbf{H})\boldsymbol{\mu} \cdot (\mathbf{I}_n - \mathbf{H})\boldsymbol{\epsilon}$$

has zero expectation because

$$E(\boldsymbol{\epsilon} \mid \mathbf{X}) = 0.$$

Thus,

$$\begin{aligned} E(\text{MSE}_{\text{train}} \mid \mathbf{X}) &= \frac{1}{n} \left[\|(\mathbf{I}_n - \mathbf{H})\boldsymbol{\mu}\|^2 + E(\boldsymbol{\epsilon}^\top (\mathbf{I}_n - \mathbf{H})\boldsymbol{\epsilon}) \right] \\ &= \frac{B^2}{n} + \frac{\sigma^2}{n} \text{tr}(\mathbf{I}_n - \mathbf{H}) \\ &= \frac{B^2}{n} + \frac{\sigma^2}{n} (n - (p + 1)) \\ &= \boxed{\frac{B^2}{n} + \sigma^2 \left(1 - \frac{p + 1}{n} \right)}. \end{aligned}$$

(2) Expected Test MSE

We have

$$\begin{aligned} \mathbf{y}^* - \mathbf{H}\mathbf{y} &= \boldsymbol{\mu} + \boldsymbol{\epsilon}^* - \mathbf{H}(\boldsymbol{\mu} + \boldsymbol{\epsilon}) \\ &= (\mathbf{I}_n - \mathbf{H})\boldsymbol{\mu} + \boldsymbol{\epsilon}^* - \mathbf{H}\boldsymbol{\epsilon}. \end{aligned}$$

All cross terms have zero expectation because $\boldsymbol{\epsilon}$ and $\boldsymbol{\epsilon}^*$ are independent and have mean zero.

Thus,

$$\begin{aligned} E(\text{MSE}_{\text{test}} \mid \mathbf{X}) &= \frac{1}{n} [B^2 + E(\|\boldsymbol{\epsilon}^*\|^2) + E(\|\mathbf{H}\boldsymbol{\epsilon}\|^2)] \\ &= \frac{B^2}{n} + \frac{n\sigma^2}{n} + \frac{\sigma^2 \text{tr}(\mathbf{H})}{n} \\ &= \frac{B^2}{n} + \sigma^2 + \frac{(p + 1)\sigma^2}{n} \\ &= \boxed{\frac{B^2}{n} + \sigma^2 \left(1 + \frac{p + 1}{n} \right)}. \end{aligned}$$

(3) Expected Optimism

The expected optimism is

$$\begin{aligned} E(\text{MSE}_{\text{test}} - \text{MSE}_{\text{train}} \mid \mathbf{X}) &= \left(1 + \frac{p + 1}{n} \right) \sigma^2 - \left(1 - \frac{p + 1}{n} \right) \sigma^2 \\ &= \boxed{\frac{2(p + 1)\sigma^2}{n}}. \end{aligned}$$

2. Numerical Computation for $n = 120, p = 4, \frac{B^2}{n} = 0.04$, and $\sigma^2 = 1$

Here,

$$p + 1 = 5.$$

Expected Training MSE

$$\begin{aligned}E(\text{MSE}_{\text{train}}) &= 0.04 + \left(1 - \frac{5}{120}\right) \times 1 \\&= 0.04 + \frac{115}{120} \\&= 0.04 + 0.9583 \\&= \boxed{0.9983}.\end{aligned}$$

Expected Test MSE

$$\begin{aligned}E(\text{MSE}_{\text{test}}) &= 0.04 + \left(1 + \frac{5}{120}\right) \times 1 \\&= 0.04 + \frac{125}{120} \\&= 0.04 + 1.0417 \\&= \boxed{1.0817}.\end{aligned}$$

Expected Difference (Optimism)

$$\begin{aligned}E(\text{MSE}_{\text{test}} - \text{MSE}_{\text{train}}) &= \frac{2 \times 5 \times 1}{120} \\&= \frac{10}{120} \\&= \boxed{0.0833}.\end{aligned}$$

Why the Ordering Need Not Hold for Every Realized Pair

The above values are **expectations**, meaning they are averages over many repeated samples.

For a single realized pair of training and test responses:

- The training noise ϵ might be unusually small, making the realized training MSE much lower than its expectation.
- The test noise ϵ^* might be unusually large, making the realized test MSE much higher than its expectation.
- It is therefore possible that, for one particular realization,

$$\text{MSE}_{\text{train}} > \text{MSE}_{\text{test}}.$$

Only after averaging over many realizations does the positive expected optimism

$$\frac{2(p+1)\sigma^2}{n}$$

become apparent.

3. Given $n = 120$, $p = 4$, $\text{RSS} = 116.4$, and $\hat{\sigma}^2 = 0.96$

We have

$$p + 1 = 5.$$

(1) Corrected Test MSE

The corrected estimate of the test MSE is

$$\begin{aligned}\widehat{\text{MSE}}_{\text{test}} &= \frac{\text{RSS} + 2(p+1)\hat{\sigma}^2}{n} \\ &= \frac{116.4 + 2 \times 5 \times 0.96}{120} \\ &= \frac{116.4 + 9.6}{120} \\ &= \frac{126.0}{120} \\ &= \boxed{1.05}.\end{aligned}$$

(2) Mallows' C_p

Mallows' C_p is

$$C_p = \frac{\text{RSS}}{\hat{\sigma}^2} - n + 2(p+1).$$

Substituting the given values,

$$\begin{aligned}C_p &= \frac{116.4}{0.96} - 120 + 10 \\ &= 121.25 - 120 + 10 \\ &= \boxed{11.25}.\end{aligned}$$

(3) Relationship Between $\widehat{\text{MSE}}_{\text{test}}$ and C_p

Starting from the corrected test MSE,

$$\widehat{\text{MSE}}_{\text{test}} = \frac{\text{RSS} + 2(p+1)\hat{\sigma}^2}{n}.$$

Multiply both sides by $\frac{n}{\hat{\sigma}^2}$:

$$\begin{aligned}\frac{n}{\hat{\sigma}^2} \widehat{\text{MSE}}_{\text{test}} &= \frac{\text{RSS} + 2(p+1)\hat{\sigma}^2}{\hat{\sigma}^2} \\ &= \frac{\text{RSS}}{\hat{\sigma}^2} + 2(p+1).\end{aligned}$$

Now recall that

$$C_p = \frac{\text{RSS}}{\hat{\sigma}^2} - n + 2(p+1).$$

Therefore,

$$\begin{aligned}C_p &= \left(\frac{\text{RSS}}{\hat{\sigma}^2} + 2(p+1) \right) - n \\ &= \frac{n}{\hat{\sigma}^2} \widehat{\text{MSE}}_{\text{test}} - n.\end{aligned}$$

Hence,

$$C_p = \frac{n}{\hat{\sigma}^2} \widehat{\text{MSE}}_{\text{test}} - n.$$

Equivalently,

$$\widehat{\text{MSE}}_{\text{test}} = \frac{\hat{\sigma}^2}{n} (C_p + n).$$

(4) Why Minimizing C_p Is Equivalent to Minimizing the Corrected Test MSE

When comparing a set of candidate models:

- n is the same for all candidate models.
- $\hat{\sigma}^2$ is the same common estimate of the error variance for all candidate models.

Therefore,

$$\frac{\hat{\sigma}^2}{n}$$

is a positive constant across all candidate models.

From

$$\widehat{\text{MSE}}_{\text{test}} = \frac{\hat{\sigma}^2}{n} (C_p + n),$$

we can see that $\widehat{\text{MSE}}_{\text{test}}$ is a positive linear transformation of C_p .

Adding the constant n and multiplying by the positive constant $\frac{\hat{\sigma}^2}{n}$ do not change the ordering of the candidate models.

Therefore,

$$\arg \min C_p = \arg \min \widehat{\text{MSE}}_{\text{test}}.$$

In other words:

The model that minimizes Mallows' C_p also minimizes the corrected estimate of the test MSE.

This is the mathematical justification for using Mallows' C_p as a model selection criterion.

Question 4: Comparing Mallows' C_p , AIC, and BIC

```
In [143... def question4():
    def fit_ols_rss(var, data):
        X = np.column_stack((np.ones(n), data[var].values))
        beta_hat = np.linalg.lstsq(X, data["y"].values, rcond=None)[0]
        y_hat = X @ beta_hat
        Rss = np.sum((data["y"].values - y_hat)**2)
        return Rss

    data = pd.read_csv("../data/diabetes.csv").iloc[:370]
```



```

n = data.shape[0]
variable_A = ["bmi", "bp", "s5", "sex", "s1", "s2", "s4"]
variable_B = ["bmi", "bp", "s5", "sex", "s1", "s2", "s4", "s6"]
variable_full = ['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6']
##a
print ("a")
rss_a = fit_ols_rss(variable_A,data)
print ("Model A: p = %d, RSS = %.2f"%(len(variable_A),rss_a))
rss_b = fit_ols_rss(variable_B,data)
print ("Model B: p = %d, RSS = %.2f"%(len(variable_B),rss_b))
rss_full = fit_ols_rss(variable_full,data)
residual_df = n - (len(variable_full) + 1)
sigma2_hat = rss_full / residual_df
print ("Model Full: p = %d, RSS = %.2f, residual_df = %.2f, sigma2_hat = %.2f"%(l
##b
def crit(rss_val, p_val):
    cp = rss_val / sigma2_hat - n + 2 * (p_val + 1)
    aic = n * np.log(rss_val / n) + 2 * (p_val + 1)
    bic = n * np.log(rss_val / n) + np.log(n) * (p_val + 1)
    return cp, aic, bic
print("\nb")
for name, rss_val, p_val in [
    ("A", rss_a, len(variable_A)),
    ("B", rss_b, len(variable_B)),
]:
    cp, aic, bic = crit(rss_val, p_val)
    print("Model %s: Cp = %.4f, AIC = %.4f, BIC = %.4f"
          % (name, cp, aic, bic))

##c
print("\nc.1")
fit_gain = n * np.log(rss_a / rss_b)
cp_gain = (rss_a - rss_b) / sigma2_hat
print("fit_gain (AIC/BIC) = %.4f" % fit_gain)
print("cp_gain = %.4f" % cp_gain)
print("AIC penalty = 2, Cp penalty = 2, BIC penalty = log(n) = %.4f" % np.log(n))

```

In [144... question4()

a

Model A: p = 7, RSS = 1098908.57

Model B: p = 8, RSS = 1089717.06

Model Full: p = 10, RSS = 1089451.72, residual_df = 359.00, sigma2_hat = 3034.68

b

Model A: Cp = 8.1163, AIC = 2974.6403, BIC = 3005.9483

Model B: Cp = 7.0874, AIC = 2973.5325, BIC = 3008.7540

c.1

fit_gain (AIC/BIC) = 3.1078

cp_gain = 3.0288

AIC penalty = 2, Cp penalty = 2, BIC penalty = log(n) = 5.9135

c.2 Adding `s6` reduces the fit term by $n \cdot \log(\text{RSS}_A / \text{RSS}_B) = 3.108$ on the AIC/BIC scale, and by $(\text{RSS}_A - \text{RSS}_B) / \text{sigma2_hat} = 3.029$ on the Cp scale. AIC and Cp charge a penalty of only 2 for the one extra parameter, so the improvement ($3.108 > 2$, and $3.029 > 2$) outweighs the penalty, and they select Model B. BIC, however, charges $\log(n) = \log(370) = 5.914$ for that same extra parameter. Since $3.108 < 5.914$, the improvement does not cover the penalty, so BIC keeps the simpler Model A. The disagreement therefore comes entirely from the different penalties: AIC and Cp penalize lightly (2), while BIC penalizes more heavily ($\log n$).

c.3 Selecting a model only ranks a small set of candidate models under a particular criterion. It chooses the best model among those candidates, not the model that actually generated the data. The true data-generating model may not be in the candidate set at all, or it may involve omitted

variables, nonlinearities, or interactions that none of the fitted candidates contain. Model selection is therefore a relative comparison, not a proof of the truth.

Question 5: Validation and final test data

- a. Step 2 is the first misuse. Although the coefficients are fitted using only the training data, Steps 2-3 use the final test responses to compare and select among the candidate models. The test set therefore participates in model selection, so it is no longer an untouched final evaluation.
- b. Each of the eleven test MSEs contains random variation. Taking the minimum tends to pick the model whose test MSE happened to be unusually small by chance. Reporting that same minimum as the final estimate is then overly optimistic: the number has already been "cherry-picked" from the eleven values, so it underestimates the true prediction error of the selected procedure.
- c. A valid procedure is:
 1. Reserve rows 371-442 as the final test set and do not inspect their responses.
 2. Divide rows 1-370 into five folds.
 3. For each candidate predictor count m , train on four folds and compute the MSE on the remaining fold; repeat until every fold has served as the validation fold.
 4. Average the five validation MSEs for each m and select the m with the smallest average.
 5. Refit the selected model using all 370 training observations.
 6. Evaluate this fitted model once, only at the end, on rows 371-442.

```
In [131... def question5():
    data = pd.read_csv("../data/diabetes.csv")

    train = data.iloc[:370]
    n_train = train.shape[0]
    test = data.iloc[370:]
    variable_full = ['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6']
    kf = KFold(n_splits=5, shuffle=False)
    cv_mse = np.zeros(len(variable_full)+1)

    for m in range(len(variable_full)):
        varss = variable_full[:m]
        Xm = np.column_stack((np.ones(n_train), train[varss]))
        fold_mse = []
        for tr_idx, va_idx in kf.split(Xm):
            beta = np.linalg.lstsq(Xm[tr_idx], train["y"].values[tr_idx], rcond=None)
            pred = Xm[va_idx] @ beta
            fold_mse.append(np.mean((train["y"].values[va_idx] - pred) ** 2))
        cv_mse[m] = np.mean(fold_mse)
    m_best = int(np.argmin(cv_mse))
```

```
In [130... question5()
```

```
In [ ]:
```